

TAREFA PRINCIPAL

REESTRUTURAR E INTEGRAR O CLAMOUR + UNIVERSAL SERVER À ARQUITETURA REAL DO PLAYCANVAS

**NÃO FAZER APENAS ALTERAÇÕES VISUAIS OU EM ARQUIVOS.
GARANTIR INTEGRAÇÃO REAL DE RUNTIME.**

Você está trabalhando diretamente no projeto atualmente aberto no Play Canvas.

O projeto foi restaurado a partir de um backup e é esse estado atual que deve ser preservado.

O objetivo desta tarefa NÃO é apenas “fazer o jogo parecer PlayCanvas”.

O objetivo é fazer o jogo ser **estrutural e funcionalmente integrado ao PlayCanvas**, aproveitando o código e sistemas já existentes do Clamour e do Universal Server, sem jogar fora o trabalho existente.

1. DIAGNÓSTICO FUNDAMENTAL

O repositório Web atual é baseado em:

```
React  
Vite  
Three.js  
TypeScript
```

O `package.json` atual da raiz possui, entre outras:

```
"react": "^18.3.1",  
"react-dom": "^18.3.1",  
"three": "^0.179.1",  
"vite": "^7.1.2"
```

e scripts como:

```
"dev": "node scripts/start-stack.mjs",  
"dev:clamour": "vite --host 0.0.0.0",  
"dev:us": "pnpm --dir universal-server run start"
```

Isso significa que o runtime atual do Web Game NÃO deve ser presumido como um projeto PlayCanvas nativo simplesmente porque estamos usando uma interface chamada Play Canvas.

O Kilo deve verificar qual runtime está realmente sendo executado.

Não assumir.

2. O QUE QUEREMOS CONSTRUIR

Queremos que:

CLAMOUR WEB

seja o cliente/jogo real.

E:

UNIVERSAL SERVER

seja o backend/servidor real.

E o PlayCanvas seja efetivamente responsável pela parte de runtime de jogo que ele deve executar:

Scene
Entities
Components
Scripts
Assets
Camera
Input
Physics
Animation
UI
World
Player
Game Systems

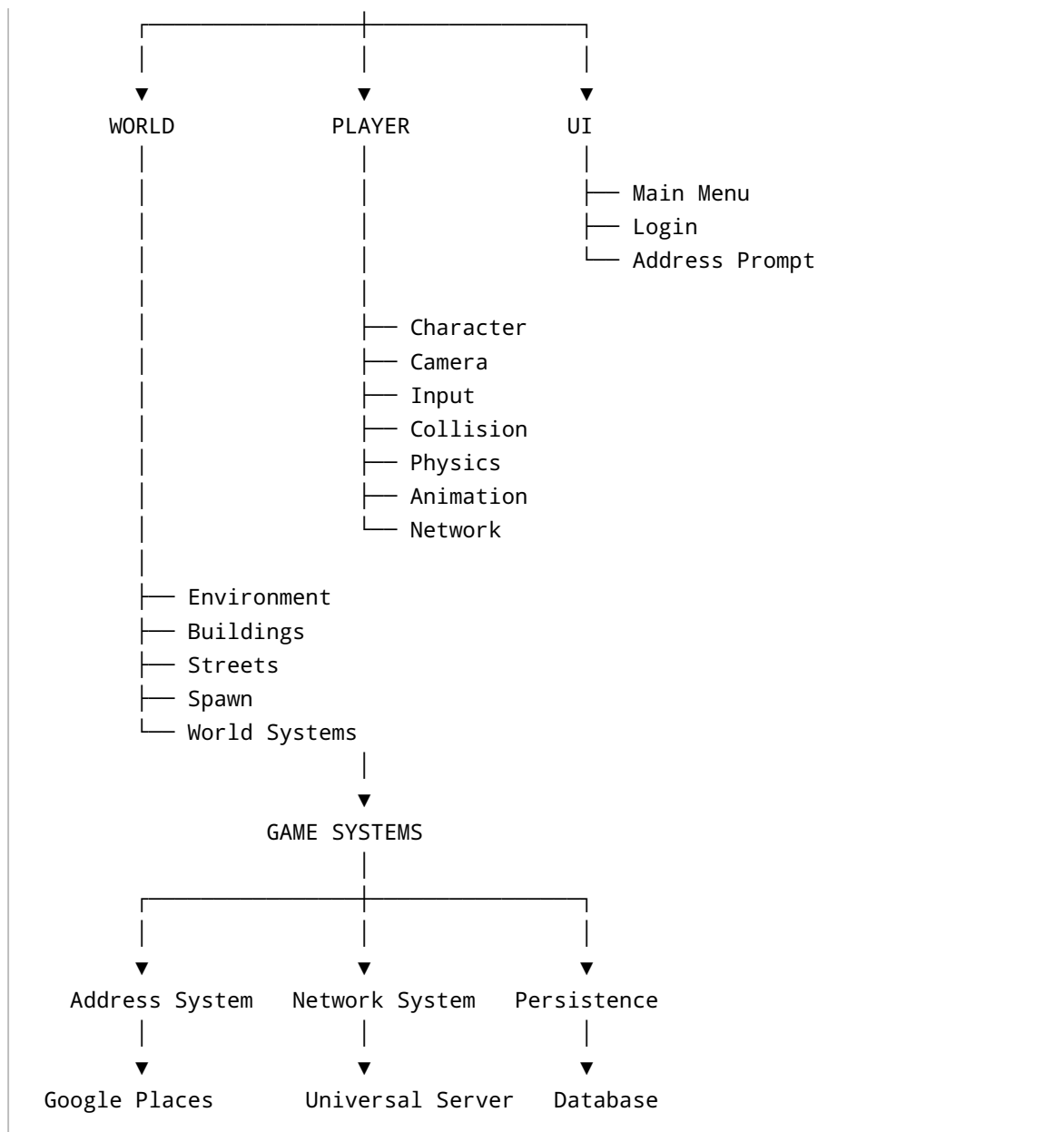
A arquitetura conceitual deve ser:

PLAYCANVAS APPLICATION



GAME SCENE





3. NÃO CONFUNDIR "ARQUIVO EXISTE" COM "SISTEMA FUNCIONA"

É obrigatório entender esta diferença:

arquivo existe
≠
sistema conectado

e:

função existe
≠
função chamada

e:

Entity existe
≠
Entity participa do gameplay

e:

Script existe
≠
Script está anexado à Entity correta

e:

Asset existe
≠
Asset foi carregado pelo runtime

e:

UI existe
≠
UI controla o jogo

e:

Universal Server responde
≠
o jogo está realmente integrado a ele

A tarefa é corrigir essas conexões.

4. ANALOGIA DO CARRO

Use esta regra como modelo de arquitetura:

Não basta existir:

volante
motor
freio
roda
eixo
painel

Cada peça precisa estar conectada.

No jogo:

AddressPrompt

deve estar conectado ao:

Address System

que deve estar conectado ao:

Google Places

que deve retornar:

Place
Latitude
Longitude

que devem alimentar:

Spawn System

que deve controlar:

Player Entity

que possui:

Transform
Character
Camera
Collision
Physics

Input
Animation
Network

e o sistema de Network comunica com:

Universal Server

Se qualquer ligação estiver faltando, o sistema não está integrado.

5. ESTUDAR E APLICAR A ARQUITETURA REAL DO PLAYCANVAS

Use a documentação oficial atual do PlayCanvas como referência.

Conceitos obrigatórios:

Entity
Component
Script
Asset
Scene
Scene Graph
Hierarchy
Transform
Reference
Component System
Script Component
Event System
Manager/System
Prefab/Template
Instantiation
Runtime
Game State
Input
Physics
Animation
UI
Networking
Persistence
Dependency Graph
Data Flow

O PlayCanvas utiliza uma arquitetura ECS:

```
Entity
+
Components
+
Systems
```

Entidades são contêineres na cena; componentes fornecem capacidades; systems gerenciam os componentes.

6. APLICAÇÃO / RUNTIME

Se estivermos usando PlayCanvas React:

deve existir um:

```
<Application>
```

que inicializa o runtime do Engine.

Dentro dele devem existir as entidades/componentes reais do jogo.

Não criar apenas uma página React que desenha uma interface e chamar isso de PlayCanvas.

7. ENTITY

Use Entities reais para elementos do gameplay.

Exemplos:

```
GameRoot
World
Player
Camera
SpawnManager
NetworkManager
GameManager
AddressManager
UI
```

Uma Entity pode ter filhos:

```
Player
├─ Model
├─ Camera
├─ Collider
└─ Detection
```

A hierarquia é funcional porque transformações parentais afetam os filhos.

8. COMPONENTS

Os componentes devem ser utilizados de acordo com a função real.

Exemplos:

```
Player
├─ Render
├─ Collision
├─ Rigidbody
└─ Script

Camera
└─ Camera

MainUI
└─ Screen

AddressPrompt
├─ Screen
├─ Element
└─ Script
```

O PlayCanvas possui componentes para câmera, render, collision, rigidbody, animation, UI, scripts e vários outros sistemas.

Não criar componentes apenas para preencher a árvore.

Cada componente deve ter propósito.

9. SCRIPTS

Os scripts devem estar realmente associados às entidades.

No PlayCanvas moderno, ESM (`.mjs`) é o sistema recomendado para novos scripts.

Um script deve estar:

```
Script Asset
  ↓
Script Component
  ↓
Entity
  ↓
Runtime
```

O script não pode ser apenas um arquivo solto dentro do projeto.

Os scripts devem utilizar corretamente:

```
this.app
this.entity
```

quando apropriado.

10. SCRIPT ATTRIBUTES

Quando uma referência precisa ser configurada no Editor, preferir Script Attributes.

Exemplo:

```
/**
 * @attribute
 * @type {Entity}
 */
target;
```

Isso permite que o Editor mantenha uma referência real a outra Entity.

Isso é muito importante para evitar referências frágeis baseadas somente em nomes pesquisados manualmente.

Use atributos para:

```
Player
Camera
SpawnPoint
UI
```

```
GameManager
NetworkManager
```

quando apropriado.

11. ASSETS

Os Assets do jogo precisam estar corretamente organizados no Asset Panel.

Organizar por pastas.

Estrutura sugerida:

```
assets/
├─ characters/
├─ environments/
├─ buildings/
├─ vehicles/
├─ animations/
├─ materials/
├─ textures/
├─ audio/
├─ UI/
├─ scripts/
├─ prefabs/
└─ scenes/
```

O PlayCanvas gerencia Assets no Asset Panel e permite organizar esses recursos por hierarquia de pastas.

Não deixar dezenas de arquivos misturados sem organização.

12. MODELOS 3D

Quando um modelo possui hierarquia interna, utilizar a hierarquia de importação/Template de maneira correta.

O PlayCanvas pode importar uma hierarquia de modelo como entidades e gerar elementos como:

```
Template
Container
Render
```

Materials
Textures

conforme a pipeline do asset.

Portanto:

Player

deve poder ser um elemento estruturado, não apenas uma imagem solta.

13. SCENE

Criar ou organizar a Scene real do jogo.

A Scene deve conter:

Entities
Components
Settings
World
Player
Camera
Lighting
UI
Systems

Uma Scene é a fundação do aplicativo PlayCanvas e contém a hierarquia das Entities e suas configurações.

A Scene que o runtime carrega deve ser a Scene que contém o jogo real.

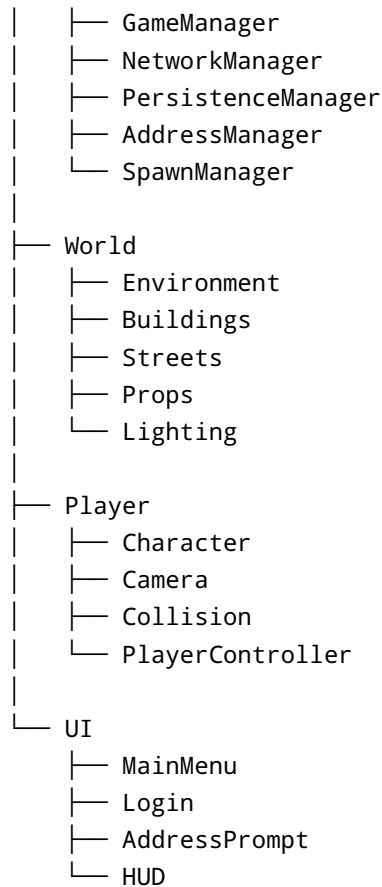
Não criar uma Scene bonita no Editor que nunca é carregada.

14. SCENE GRAPH

A hierarquia deve ter propósito.

Sugestão:

Root
|
├─ Systems



Essa é uma referência arquitetural.

Adapte aos objetos reais já existentes no projeto.

15. GAME MANAGER

Criar/identificar um:

GameManager

responsável por coordenar:

boot
initialization
game state
scene state
player state
network connection

Não colocar toda a lógica em um único script gigante.

16. NETWORK MANAGER

Criar/identificar:

NetworkManager

Responsabilidade:

Clamour
↓
NetworkManager
↓
Universal Server

Esse sistema deve controlar:

connect
disconnect
authentication
session
player state
events
multiplayer

quando suportado pelos APIs existentes.

17. ADDRESS MANAGER

Criar/identificar:

AddressManager

Responsabilidade:

AddressPrompt
↓
Places Autocomplete
↓
Place Details
↓
Location

↓
Spawn

O AddressPrompt não deve ser responsável por toda a arquitetura.

Ele é a UI.

O AddressManager é o sistema.

18. GOOGLE PLACES

O fluxo obrigatório é:

```
usuário digita
↓
Places Autocomplete
↓
suggestions
↓
usuário seleciona
↓
placeId
↓
Place Details
↓
formattedAddress
latitude
longitude
↓
Spawn
```

Não usar Geocoding como falso autocomplete.

19. UNIVERSAL SERVER

O Universal Server deve continuar sendo o backend REAL.

Ele deve continuar responsável pelas APIs reais.

Não criar mock.

Não criar backend fake para o PlayCanvas.

O PlayCanvas deve conversar com o Universal Server real.

Arquitetura:

```
PlayCanvas
↓
NetworkManager
↓
Universal Server
↓
database / Google / services
```

20. PERSISTENCE

O sistema de persistência existente deve continuar funcionando.

Exemplo:

```
Player
↓
PlayerState
↓
NetworkManager
↓
Universal Server
↓
Persistence
```

Não usar somente:

```
localStorage
```

como substituto do backend.

21. UI

A UI que já existe no Clamour não deve simplesmente continuar sendo uma tela React independente se a intenção é que ela faça parte do runtime PlayCanvas.

Mapear:

```
Main Menu
Login
Address Prompt
```

HUD
Menus
Dialogs

para o sistema de UI que efetivamente será usado pelo runtime escolhido.

No PlayCanvas, UI pode ser estruturada com `Screen`, `Element`, `Button`, `Scroll View`, etc.

22. NÃO DUPLICAR UI

Não manter:

React AddressPrompt

e:

PlayCanvas AddressPrompt

fazendo a mesma coisa sem necessidade.

Definir uma única fonte oficial para o runtime do jogo.

Se React for mantido como camada de UI, provar que essa camada está realmente integrada ao PlayCanvas Runtime.

Se o PlayCanvas for responsável pela UI, mover/conectar a UI adequadamente.

Não deixar duas versões divergentes.

23. PLAYER

O Player deve existir como Entity REAL.

Estrutura conceitual:

```
Player
├─ Transform
├─ Render
├─ Collision
├─ Rigidbody
└─ Script
```



```
├ Camera
└ Network
```

Os componentes exatos dependem da implementação física real.

24. INPUT

O sistema de input deve realmente controlar o Player.

Fluxo:

```
Keyboard / Mouse / Touch / Gamepad
    ↓
Input System
    ↓
PlayerController
    ↓
Player Entity
    ↓
Physics / Animation
```

Não ter controles implementados somente em React enquanto o Player vive em outro runtime, sem uma ponte funcional.

25. PHYSICS

Se houver física:

```
Player
    ↓
Collision
    ↓
RigidBody
    ↓
Physics System
```

As colisões devem realmente afetar o movimento.

26. CAMERA

A câmera deve ser uma Entity com:

Camera Component

e estar realmente ligada ao Player/controlador.

Não pode existir uma câmera no arquivo e outra câmera independente renderizando a cena.

27. ANIMATION

O sistema de animação deve estar ligado ao personagem:

```
Player
↓
Character
↓
Anim Component
↓
Animation State
```

O PlayCanvas possui componente `Anim` para state graphs/animations.

28. EVENT SYSTEM

Sistemas devem se comunicar por eventos quando apropriado.

Exemplo:

```
Address Selected
↓
location:selected
↓
SpawnManager
```

Outro exemplo:

```
Player Connected
↓
player:connected
↓
UI / Network / GameManager
```

O objetivo é reduzir ligações frágeis e manter a arquitetura compreensível.

29. PREFABS / TEMPLATES

Elementos reutilizáveis devem ser transformados em templates/prefabs.

Exemplos:

```
Player.prefab  
Enemy.prefab  
Vehicle.prefab  
Building.prefab  
UIWindow.prefab
```

Depois instanciar quando necessário.

Não duplicar manualmente a mesma estrutura dezenas de vezes.

30. RUNTIME

Tudo precisa existir no Runtime REAL.

Isso significa:

```
Editor hierarchy  
  ↓  
Scene  
  ↓  
Launch  
  ↓  
Application  
  ↓  
entities/components/scripts/asset
```

O que está no Editor precisa ser efetivamente carregado/instanciado pela aplicação.

O que está no código precisa realmente ser chamado.

31. VERIFICAÇÃO DE RUNTIME

Não considerar a tarefa concluída somente olhando os arquivos.

Depois das alterações:

1. abrir a Scene;
2. verificar a Hierarchy;
3. verificar as Entities;
4. verificar os Components no Inspector;
5. verificar referências;
6. iniciar Launch;
7. observar a aplicação;
8. verificar logs;
9. testar interação;
10. verificar estado das Entities.

A própria documentação do PlayCanvas recomenda esse tipo de verificação de runtime por meio de Editor/Launch e inspeção do estado da aplicação.

32. COMPARAR A INTERFACE

IMPORTANTE:

Compare a interface atual que aparece no jogo com:

```
a interface desejada
+
o código existente
+
a Scene do PlayCanvas
```

Não assumir que a tela atualmente renderizada é a tela correta.

Descobrir:

```
quem está renderizando
quem controla
quem recebe input
quem dispara os eventos
```

Se for React:

documentar.

Se for Three.js:

documentar.

Se for PlayCanvas:

documentar.

Se houver mistura:

mapear as pontes entre eles.

33. DIAGNÓSTICO OBRIGATÓRIO DO RUNTIME ATUAL

Antes da migração/integração:

identificar:

```
Renderer atual:  
Three.js / PlayCanvas / outro
```

```
UI atual:  
React / DOM / PlayCanvas
```

```
World:  
Three / PlayCanvas
```

```
Player:  
Three / PlayCanvas
```

```
Camera:  
Three / PlayCanvas
```

```
Input:  
...
```

```
Physics:  
...
```

```
Networking:  
...
```

```
Persistence:  
...
```

```
Google:  
...
```

```
Universal Server:  
...
```

Entregar esse mapa internamente antes de modificar.

34. NÃO JOGAR FORA THREE.JS SEM NECESSIDADE

O fato de o projeto utilizar Three.js não significa automaticamente que ele deve ser apagado.

Primeiro entender qual parte do projeto depende dele.

Se houver código útil:

```
gameplay  
models  
world generation  
camera  
effects
```

avaliar como integrar/migrar.

Não apagar simplesmente porque PlayCanvas foi escolhido.

35. NÃO MANTER DOIS RUNTIMES CONCORRENTES SEM ARQUITETURA

Não deixar:

```
Three.js renderizando o jogo  
+  
PlayCanvas renderizando outro jogo
```

ao mesmo tempo sem uma razão arquitetural explícita.

Isso criaria duas engines competindo pela responsabilidade do jogo.

Definir quem é o runtime oficial.

36. SE PLAYCANVAS FOR O RUNTIME OFICIAL

Então:

```
PlayCanvas Application
```

deve ser a autoridade do gameplay.

React deve ser:

```
camada administrativa  
ou  
UI auxiliar
```

somente se realmente necessária e conectada.

O mundo, Player, câmera, input e gameplay principal devem existir no runtime oficial.

37. SE PLAYCANVAS REACT FOR O RUNTIME OFICIAL

Usar corretamente:

```
<Application>  
<Entity>  
<Component>
```

e os hooks apropriados.

O `Application` fornece o contexto do Engine e o `Entity` é o bloco fundamental da cena.

38. SCRIPTS DEVE SEREM REUTILIZÁVEIS E CONFIGURÁVEIS

Quando possível:

```
PlayerController  
CameraController  
SpawnManager  
NetworkManager  
AddressManager  
GameManager
```

devem ser scripts reutilizáveis.

A configuração específica deve ir para:

Script Attributes

quando isso melhorar a manutenção.

39. ESTRUTURA DE PASTAS

Organizar o projeto de acordo com responsabilidades.

Por exemplo:

```
src/  
├── core/  
│   ├── game/  
│   ├── networking/  
│   ├── persistence/  
│   └── services/  
├── gameplay/  
│   ├── player/  
│   ├── world/  
│   ├── spawning/  
│   ├── enemies/  
│   └── vehicles/  
├── google/  
│   ├── places/  
│   ├── geocoding/  
│   └── streetview/  
├── ui/  
│   ├── menu/  
│   ├── login/  
│   ├── address/  
│   └── hud/  
└── playcanvas/  
    ├── entities/  
    ├── scripts/  
    ├── systems/  
    └── scenes/
```

Adapte ao projeto real.

Não criar pastas apenas por estética.

40. UNIVERSAL SERVER E PLAYCANVAS

A ligação deve ser explícita:

```
PlayCanvas NetworkManager
    ↓
Universal Server API
    ↓
Authentication
    ↓
Player State
    ↓
Persistence
```

E:

```
AddressManager
    ↓
Universal Server
    ↓
Google Places
```

41. GOOGLE KEY

A chave:

```
GOOGLE_MAPS_API_KEY
```

permanece no Universal Server.

Não enviar a secret ao runtime do browser.

O PlayCanvas deve chamar o backend.

42. CHECKPOINTS REAIS

Criar checkpoints de arquitetura:

CHECKPOINT 1
PlayCanvas Application inicia

CHECKPOINT 2
Scene carrega

CHECKPOINT 3
Player Entity existe

CHECKPOINT 4
Camera está conectada

CHECKPOINT 5
Input controla Player

CHECKPOINT 6
World renderiza

CHECKPOINT 7
Address UI aparece

CHECKPOINT 8
Google Places responde

CHECKPOINT 9
Spawn recebe lat/lon

CHECKPOINT 10
NetworkManager conecta ao US

CHECKPOINT 11
PlayerState é salvo

CHECKPOINT 12
Gameplay funciona

43. TESTE COMPLETO

Testar:

```
abrir aplicação  
↓  
menu  
↓  
login  
↓  
address prompt
```

```
↓  
digitar endereço  
↓  
Google suggestions  
↓  
selecionar endereço  
↓  
Place Details  
↓  
lat/lon  
↓  
spawn  
↓  
player control  
↓  
camera  
↓  
collision  
↓  
network  
↓  
persistence
```

44. TESTE DA INTERFACE

A aparência visual também precisa ser comparada.

Não basta:

```
"tem uma caixa de endereço"
```

Verificar:

```
posição  
tamanho  
fontes  
hierarquia  
botões  
responsividade  
z-index  
input  
dropdown  
interações
```

E principalmente:

a interface exibida é realmente a interface conectada ao runtime?

45. NÃO CRIAR UMA DEMO PARALELA

É proibido criar um pequeno projeto PlayCanvas de demonstração separado e declarar que a migração foi feita.

A Scene do PlayCanvas deve representar o jogo real.

Os sistemas reais devem estar conectados.

46. NÃO DECLARAR "PRONTO" PORQUE O CÓDIGO COMPILA

Compilar:

OK

não significa:

integrado

Precisamos:

```
build
+
scene
+
entities
+
components
+
scripts
+
assets
+
runtime
+
network
```

+
persistence

funcionando juntos.

47. DOCUMENTAÇÃO OFICIAL A SER USADA

Basear as decisões na documentação oficial atual:

PlayCanvas Engine
PlayCanvas Editor
Entities
Components
Scenes
Hierarchy
Scripting
ESM Scripts
Script Attributes
Assets
Templates
PlayCanvas React
MCP Server / Editor tooling

Fontes oficiais:

<https://developer.playcanvas.com/user-manual/engine/>
<https://developer.playcanvas.com/user-manual/ecs/>
<https://developer.playcanvas.com/user-manual/editor/>
<https://developer.playcanvas.com/user-manual/editor/scenes/>
<https://developer.playcanvas.com/user-manual/editor/scenes/components/>
<https://developer.playcanvas.com/user-manual/editor/scripting/>
<https://developer.playcanvas.com/user-manual/react/>

Use a documentação atual, não exemplos antigos quando houver API moderna equivalente.

48. NÃO USAR PADRÕES ANTIGOS SEM NECESSIDADE

Para scripts novos, priorizar:

ESM Scripts
.mjs

que são o sistema recomendado atualmente.

Não introduzir Classic Scripts apenas por hábito.

49. EDITOR + CÓDIGO PRECISAM SER CONSISTENTES

A arquitetura correta é:

Código
↓
Scene
↓
Entities
↓
Components
↓
Script Attributes
↓
References
↓
Assets
↓
Runtime

Se o código espera:

Player

mas a cena possui:

PlayerCharacter

corrigir a referência.

Se o script espera:

Camera

mas nenhuma câmera está conectada:

corrigir.

Se o código espera um asset:

```
player.glb
```

mas a cena não tem esse asset:

corrigir.

50. RESULTADO ARQUITETURAL ESPERADO

No final, quero poder abrir o PlayCanvas Editor e visualizar uma estrutura organizada semelhante a:

```
ROOT
|
├── SYSTEMS
|   ├── GameManager
|   ├── NetworkManager
|   ├── AddressManager
|   ├── SpawnManager
|   └── PersistenceManager
|
├── WORLD
|   ├── Environment
|   ├── Streets
|   ├── Buildings
|   ├── Lighting
|   └── Props
|
├── PLAYER
|   ├── Character
|   ├── Camera
|   ├── Collision
|   └── Controller
|
└── UI
    ├── MainMenu
    ├── Login
    ├── AddressPrompt
    └── HUD
```

Cada uma dessas Entities deve ter componentes e scripts apropriados.

Não apenas nomes bonitos na árvore.

51. CRITÉRIO DE FUNCIONALIDADE

Cada sistema deve poder ser traçado:

```
INPUT
↓
SCRIPT
↓
ENTITY
↓
COMPONENT
↓
SYSTEM
↓
WORLD / PLAYER
↓
NETWORK
↓
SERVER
```

E para endereço:

```
UI
↓
AddressManager
↓
Universal Server
↓
Google Places
↓
Place Details
↓
lat/lon
↓
SpawnManager
↓
Player Entity
```

52. TESTE DE INTEGRAÇÃO REAL

Fazer pelo menos estes testes:

- A. abrir cena
- B. iniciar runtime
- C. verificar câmera
- D. verificar player
- E. verificar input
- F. verificar UI
- G. selecionar endereço
- H. verificar Google
- I. verificar spawn
- J. verificar conexão com US
- K. verificar persistência

53. FERRAMENTAS DO PLAYCANVAS

Quando houver capacidade de inspeção/edição via tooling do PlayCanvas:

usar isso para:

```
Entities  
Components  
Scripts  
Assets  
Scenes  
Templates  
References  
Attributes
```

E depois verificar o resultado visual e de runtime.

A documentação oficial indica que as ferramentas de Editor podem inspecionar e editar entidades, componentes, scripts, assets, scenes, templates e estado de execução.

54. NÃO PARAR NO CÓDIGO

É obrigatório validar:

```
Código  
+  
Editor  
+  
Runtime
```

Os três.

55. NÃO APAGAR FUNCIONALIDADES EXISTENTES

Preservar:

```
Google
Street View
Geocoding
Places
Universal Server
Authentication
Persistence
Player State
Game logic
```

Migrar/conectar.

Não remover.

56. NÃO ALTERAR SECRET MANAGEMENT

Não tocar em:

```
.env
secrets
API keys
tokens
```

fora do necessário para conectar o runtime.

Nunca expor secrets.

57. RESULTADO FINAL

O que quero é um jogo onde:

```
PlayCanvas
  ↓
Application
  ↓
Scene
  ↓
```

```
Entities
  ↓
Components
  ↓
Scripts
  ↓
Assets
  ↓
Game Systems
  ↓
Clamour gameplay
  ↓
Universal Server
```

seja uma única arquitetura funcional.

Não quero:

```
arquivo solto
+
função solta
+
UI solta
+
backend solto
+
PlayCanvas solto
```

Quero tudo conectado.

58. REGRA FINAL

Antes de dizer que terminou, pergunte internamente para cada parte:

```
"Quem cria isso?"
"Quem controla isso?"
"Quem chama isso?"
"Quem recebe seus dados?"
"Qual Entity representa isso?"
"Qual Component dá essa capacidade?"
"Qual Script executa isso?"
"Qual Asset é usado?"
"Qual referência conecta as partes?"
"Em qual Scene isso existe?"
"Issó está realmente ativo no Runtime?"
```

Se alguma resposta for:

```
"não sei"  
"existe em um arquivo"  
"está no código mas não está conectado"  
"o React faz isso separado"  
"o Three faz isso separado"  
"o PlayCanvas tem mas não é usado"
```

a integração ainda não está concluída.

59. ENTREGA FINAL

Depois de implementar:

mostrar:

```
RUNTIME OFICIAL:  
...  
  
SCENE PRINCIPAL:  
...  
  
ENTITIES PRINCIPAIS:  
...  
  
COMPONENTES:  
...  
  
SCRIPTS:  
...  
  
ASSETS:  
...  
  
REFERÊNCIAS:  
...  
  
GAME SYSTEMS:  
...  
  
CLAMOUR → PLAYCANVAS:  
...  
  
PLAYCANVAS → UNIVERSAL SERVER:  
...
```

UNIVERSAL SERVER → GOOGLE:

...

ADDRESS → SPAWN:

...

NETWORK:

...

PERSISTENCE:

...

BUILD:

OK / ERRO

RUNTIME:

OK / ERRO

INTEGRAÇÃO:

OK / ERRO

E só declarar concluído quando:

```
Editor
+
Scene
+
Entities
+
Components
+
Scripts
+
Assets
+
Runtime
+
Clamour
+
Universal Server
```

estiverem efetivamente integrados e funcionando como um único jogo.

PRINCÍPIO CENTRAL

Não quero que o projeto apenas contenha código relacionado ao PlayCanvas. Quero que o jogo realmente execute através de uma arquitetura PlayCanvas conectada, em que as entidades, componentes, scripts, assets, cenas, referências, sistemas, UI, player, input, física, rede, Google e Universal Server participem de um fluxo funcional de runtime.